

گزارش مقایسه‌ای فنی: Rust در برابر Go برای ساخت میکروسرویس‌های مقیاس‌پذیر

گزارش فنی · شهریور ۱۴۰۵ · تهیه‌شده توسط Perplexity Computer

خلاصه اجرایی

هر دو زبان برای میکروسرویس‌های مدرن انتخاب‌های معتبری هستند، اما فلسفه‌شان متفاوت است:

• **Go** روی «سرعت تیم» بهینه شده است: کامپایل سریع، مدل همروندی ساده، زباله‌روب خودکار، زمان رسیدن به تولید کوتاه.

• **Rust** روی «قطعیت رفتار در زمان اجرا» بهینه شده است: بدون زباله‌روب، بدون توقف‌های ناگهانی GC، مصرف حافظه قابل پیش‌بینی، تضمین ایمنی حافظه در زمان کامپایل.

قاعده عملی: اگر گلوگاه شما **زمان توسعه و تعداد سرویس‌ها** است، Go را انتخاب کنید. اگر گلوگاه شما **دنباله تأخیر (p99/p999)**، **هزینه حافظه در مقیاس**، یا **کار سنگین CPU** است، Rust را انتخاب کنید.

۱. عملکرد (Performance)

تصویر کلی

در بارهای CPU-محور (سریال‌سازی سنگین، رمزنگاری، پردازش داده، فشرده‌سازی) Rust به‌طور مداوم جلوتر است چون به کد بومی بدون لایه زمان‌اجرا (runtime) و بدون GC کامپایل می‌شود. در بارهای معمول I/O-محور (سرویس REST که با دیتابیس حرف می‌زند)، اختلاف throughput اغلب کوچک می‌شود چون گلوگاه واقعی شبکه و دیتابیس است.

نکته مهم: تفاوت اصلی معمولاً در **میانگین** نیست، در **دنباله توزیع تأخیر** است.

مطالعه موردی معتبر: Discord

Discord سرویس Read States خود را از Go به Rust منتقل کرد. مشاهدات منتشرشده آن‌ها ([Discord Engineering](#)):

• نسخه Go تقریباً **هر ۲ دقیقه** یک جهش تأخیر و CPU داشت، چون Go حتی بدون رشد heap هم دست‌کم هر دو دقیقه یک چرخه GC اجباری اجرا می‌کند.

• زباله‌روب مجبور بود کل کش LRU را پیمایش کند تا مشخص شود چه چیزی آزاد است.

• کوچک‌کردن کش، جهش‌های GC را کم می‌کرد اما **تأخیر صدک ۹۹** را بدتر می‌کرد، چون نرخ اصابت کش پایین می‌آمد و باید از دیتابیس خوانده می‌شد.

- نسخه Rust حتی با بهینه‌سازی ابتدایی، از نسخه دستی‌بهینه‌شده Go بهتر عمل کرد و هیچ جهش تأخیری نداشت.
- پس از افزایش ظرفیت کش به ۸ میلیون رکورد، میانگین زمان پاسخ در حد میکروثانیه و بدترین حالت در حد میلی‌ثانیه بود.

این دقیقاً الگوی کلاسیک است: Go در حالت عادی سریع است، اما وقتی heap بزرگ و پر از اشاره‌گر دارید، هزینه GC خود را در دنباله تأخیر نشان می‌دهد.

هزینه واقعی GC در Go

طبق تیم Go (وبلاگ Go درباره Green Tea):

- برنامه‌ها می‌توانند ۲۰٪ یا بیشتر از زمان CPU خود را در زباله‌روب صرف کنند.
- حدود ۹۰٪ هزینه GC مربوط به فاز mark است و ۱۰٪ مربوط به sweep.
- از زمان mark، معمولاً دست‌کم ۳۵٪ صرف انتظار برای دسترسی به حافظه heap می‌شود؛ دسترسی به حافظه اصلی می‌تواند تا ۱۰۰ برابر کندتر از دسترسی به کش CPU باشد.

خبر خوب برای Go: زباله‌روب جدید **Green Tea** که در Go 1.25 به‌صورت آزمایشی با `GOEXPERIMENT=greenteagc` عرضه شد و طبق برنامه تیم Go قرار بود در Go 1.26 پیش‌فرض شود ([Go Blog](#))، به‌جای دنبال کردن تک‌تک اشاره‌گرها، spanهای حافظه را به‌صورت دسته‌ای اسکن می‌کند و رفتار کش را بهتر می‌کند. اندازه‌گیری‌های مستقل روی یک بار کاری پیمایش heap، کاهش زمان اجرا از حدود ۴.۲۳ ثانیه به ۲.۷۰ ثانیه (حالت packed) و از ۱۱.۰۵ ثانیه به ۶.۹۶ ثانیه (حالت scattered) را نشان داده‌اند ([The Consensus](#)).

نتیجه: شکاف عملکردی Go با Rust در حال کم‌شدن است، اما ماهیت مسئله (وجود GC) تغییر نکرده است.

جمع‌بندی عملکرد

معیار	Rust	Go
Throughput در بار CPU-محور	بالا، معمولاً چند برابر	خوب اما پایین‌تر
Throughput در بار I/O-محور	بالا	بسیار نزدیک به Rust
تأخیر p50	هر دو عالی	هر دو عالی
تأخیر p99/p999	بسیار پایدار، بدون توقف GC	مستعد جهش هنگام چرخه GC
زمان راه‌اندازی (cold start)	بسیار سریع، بدون runtime	سریع، اما runtime و GC باید گرم شوند
قابلیت پیش‌بینی زیر فشار	بالا	متوسط، وابسته به تنظیم GC

۲. مدیریت حافظه (Memory Management)

Go: زباله‌روب همزمان و قابل تنظیم

Go از یک زباله‌روب همزمان و غیرجابه‌جاکننده استفاده می‌کند؛ بیشتر کار GC همزمان با اجرای برنامه انجام می‌شود و توقف‌های stop-the-world کوتاه‌اند و متناسب با اندازه heap رشد نمی‌کنند (راهنمای رسمی GC در Go).

دو اهرم اصلی تنظیم:

- **GOGC**: تعیین می‌کند heap چقدر نسبت به داده زنده رشد کند تا GC فعال شود. مقدار پیش‌فرض ۱۰۰ یعنی وقتی heap دو برابر داده زنده شد. فرمول: $next_gc = live_heap \times (1 + GOGC/100)$. افزایش آن یعنی GC کمتر و حافظه بیشتر.

- **GOMEMLIMIT**: سقف نرم حافظه برای کل فرآیند. در محیط Kubernetes با محدودیت حافظه، این تنظیم برای جلوگیری از OOMKill حیاتی است.

الگوی رایج در تولید: `GOGC=off` یا مقدار بالا به همراه `GOMEMLIMIT` نزدیک به سقف کانتینر، تا GC فقط وقتی واقعاً لازم است اجرا شود.

منابع تأخیر ناشی از GC طبق مستندات رسمی: توقف‌های کوتاه بین فاز mark و sweep، تصاحب حدود ۲۵٪ از CPU در فاز mark، کمک اجباری goroutine‌ها به GC هنگام نرخ تخصیص بالا، هزینه اضافی نوشتن اشاره‌گرها (write barrier)، و توقف goroutine‌ها برای اسکن ریشه‌ها.

Rust: مالکیت، قرض‌گیری و آزادسازی قطعی

Rust هیچ GC ندارد. حافظه با مدل **Ownership / Borrowing / Lifetimes** مدیریت می‌شود و کامپایلر در زمان کامپایل تضمین می‌کند که:

- هر مقدار دقیقاً یک مالک دارد؛ با خروج مالک از scope، حافظه فوراً آزاد می‌شود (RAII).
- یا یک ارجاع قابل تغییر (`&mut`) وجود دارد، یا چند ارجاع فقط‌خواندنی (`&`) – نه هر دو با هم.
- در نتیجه: بدون use-after-free، بدون double-free، بدون data race در کد safe.

پیامد عملیاتی: آزادسازی قطعی. در تجربه Discord، وقتی رکوردی از کش LRU حذف می‌شد، در Rust حافظه بلافاصله آزاد می‌شد، در حالی که در Go تا اجرای GC باقی می‌ماند.

جنبه	Rust	Go
مکانیزم	Ownership + RAII، زمان کامپایل	زباله‌روب همزمان، زمان اجرا
زمان آزادسازی	قطعی و فوری	نامعین، وابسته به چرخه GC
سربار CPU در زمان اجرا	تقریباً صفر	می‌تواند به ۲۰٪+ برسد
مصرف حافظه (RSS)	معمولاً کمتر و فشرده‌تر	معمولاً بیشتر، به‌خاطر headroom مورد نیاز GC
ایمنی حافظه	تضمین‌شده در زمان کامپایل	تضمین‌شده در زمان اجرا (با هزینه GC)
هزینه انسانی	بالا، نبرد با borrow checker	بسیار پایین
اهرم‌های تنظیم	انتخاب ساختار داده، arena، Rc / Arc / Box، تخصیص‌دهنده سفارشی	GOMEMLIMIT، GOGC، sync.Pool، کاهش تخصیص

۳. منحنی یادگیری (Learning Curve)

Go: کم‌عمق و عمداً ساده

Go با حدود ۲۵ کلیدواژه، بدون ارث‌بری، بدون جنریک پیچیده در کد روزمره، و با یک راه استاندارد برای انجام هر کار طراحی شده است. یک توسعه‌دهنده باتجربه در زبان دیگر معمولاً ظرف **چند روز** کد قابل بازبینی می‌نویسد و ظرف **۲ تا ۴ هفته** بهره‌ور می‌شود. همروندی با goroutine و channel مفهومی ساده و آموزش‌پذیر است. مزیت پنهان: کد Go دیگران هم به‌راحتی خوانده می‌شود، که برای تیم‌های بزرگ و جابه‌جایی نفرات ارزش زیادی دارد.

Rust: پرشیب اما با شکل قابل پیش‌بینی

منحنی یادگیری Rust شیب تندی دارد اما الگوبیش شناخته‌شده است؛ اکثر توسعه‌دهندگان به دو «دیوار» برخورد می‌کنند: **دیوار borrow checker** در هفته‌های ۲ تا ۴، و **دیوار async و lifetime** در ماه‌های ۲ تا ۳ ([Rustify](#)). حتی مستندات رسمی زبان این هزینه را صریحاً به‌عنوان «منحنی یادگیری» و پدیده «نبرد با borrow checker» به رسمیت می‌شناسند ([The Rust Programming Language](#)).

در مقابل، Rust برای نهمین سال پیاپی محبوب‌ترین زبان در نظرسنجی Stack Overflow بوده است ([Stack Overflow](#) Developer Survey 2025) — یعنی هزینه یادگیری بالا، اما رضایت پس از تسلط بسیار بالا.

مرحله	Go	Rust
اولین سرویس قابل اجرا	۱ تا ۳ روز	۱ تا ۲ هفته
بهره‌وری در حد تولید	۲ تا ۴ هفته	۲ تا ۴ ماه
تسلط بر async و طراحی پیشرفته	۲ تا ۳ ماه	۶ تا ۱۲ ماه
هزینه استخدام	پایین، استخر بزرگ نیروی کار	بالا، استخر کوچک‌تر و گران‌تر

۴. مثال کد: یک تسک یکسان در هر دو زبان

تسک: یک میکروسرویس HTTP که یک endpoint با متد POST به آدرس `/orders` دارد. بدنه JSON را می‌خواند، اعتبارسنجی می‌کند، مبلغ کل را محاسبه می‌کند، سفارش را در یک کش درون‌حافظه‌ای امن در برابر همروندی ذخیره می‌کند و پاسخ JSON برمی‌گرداند. همچنین یک endpoint با متد GET برای خواندن سفارش دارد.

نسخه Go

```
package main

import (
    "encoding/json"
    "log"
    "net/http"
    "strings"
    "sync"
    "time"
)

type Item struct {
    SKU      string `json:"sku"`
    Qty      int    `json:"qty"`
    UnitCost float64 `json:"unit_cost"`
}

type OrderRequest struct {
    CustomerID string `json:"customer_id"`
    Items      []Item `json:"items"`
}

type Order struct {
    ID          string    `json:"id"`
    CustomerID string    `json:"customer_id"`
    Total       float64   `json:"total"`
    CreatedAt   time.Time `json:"created_at"`
}
```

```

}

// کش درون حافظه ای امن در برابر همروندی
type Store struct {
    mu      sync.RWMutex
    orders map[string]Order
}

func NewStore() *Store {
    return &Store{orders: make(map[string]Order)}
}

func (s *Store) Put(o Order) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.orders[o.ID] = o
}

func (s *Store) Get(id string) (Order, bool) {
    s.mu.RLock()
    defer s.mu.RUnlock()
    o, ok := s.orders[id]
    return o, ok
}

func validate(r OrderRequest) error {
    if strings.TrimSpace(r.CustomerID) == "" {
        return errInvalid("customer_id is required")
    }
    if len(r.Items) == 0 {
        return errInvalid("at least one item is required")
    }
    for _, it := range r.Items {
        if it.Qty <= 0 {
            return errInvalid("qty must be positive")
        }
    }
    return nil
}

type errInvalid string

func (e errInvalid) Error() string { return string(e) }

func total(items []Item) float64 {
    var sum float64
    for _, it := range items {
        sum += float64(it.Qty) * it.UnitCost
    }
    return sum
}

func main() {
    store := NewStore()
    mux := http.NewServeMux()

```

```

mux.HandleFunc("POST /orders", func(w http.ResponseWriter, r *http.Request) {
    var req OrderRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        writeErr(w, http.StatusBadRequest, "invalid json")
        return
    }
    if err := validate(req); err != nil {
        writeErr(w, http.StatusUnprocessableEntity, err.Error())
        return
    }
    order := Order{
        ID:          newID(),
        CustomerID: req.CustomerID,
        Total:       total(req.Items),
        CreatedAt:   time.Now().UTC(),
    }
    store.Put(order)
    writeJSON(w, http.StatusCreated, order)
})

mux.HandleFunc("GET /orders/{id}", func(w http.ResponseWriter, r *http.Request) {
    o, ok := store.Get(r.PathValue("id"))
    if !ok {
        writeErr(w, http.StatusNotFound, "order not found")
        return
    }
    writeJSON(w, http.StatusOK, o)
})

srv := &http.Server{
    Addr:         ":8080",
    Handler:      mux,
    ReadTimeout:  5 * time.Second,
    WriteTimeout: 10 * time.Second,
}
log.Println("listening on :8080")
log.Fatal(srv.ListenAndServe())
}

func writeJSON(w http.ResponseWriter, code int, v any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(code)
    _ = json.NewEncoder(w).Encode(v)
}

func writeErr(w http.ResponseWriter, code int, msg string) {
    writeJSON(w, code, map[string]string{"error": msg})
}

func newID() string {
    return time.Now().UTC().Format("20060102150405.000000000")
}

```

نکات: هیچ مدیریت حافظه‌ای دستی وجود ندارد. کل فایل با کتابخانه استاندارد و بدون هیچ وابستگی خارجی کار می‌کند. مدل خطا مبتنی بر مقدار بازگشتی `error` است و انضباط آن بر عهده برنامه‌نویس است – کامپایلر شما را مجبور به مدیریت خطا نمی‌کند.

نسخه Rust (با axum و tokio)

```
# Cargo.toml
[dependencies]
axum = "0.7"
tokio = { version = "1", features = ["full"] }
serde = { version = "1", features = ["derive"] }
serde_json = "1"
time = { version = "0.3", features = ["formatting"] }
```

```
use axum::{
    extract::{Path, State},
    http::StatusCode,
    response::{IntoResponse, Response},
    routing::{get, post},
    Json, Router,
};
use serde::{Deserialize, Serialize};
use std::{collections::HashMap, sync::Arc};
use tokio::sync::RwLock;

#[derive(Debug, Deserialize)]
struct Item {
    sku: String,
    qty: i64,
    unit_cost: f64,
}

#[derive(Debug, Deserialize)]
struct OrderRequest {
    customer_id: String,
    items: Vec<Item>,
}

#[derive(Debug, Clone, Serialize)]
struct Order {
    id: String,
    customer_id: String,
    total: f64,
    created_at: String,
}

// خطاها بخشی از امضای تابع اند؛ کامپایلر مدیریتشان را الزامی می‌کند
enum ApiError {
    Invalid(&'static str),
    NotFound,
}
```

```

impl IntoResponse for ApiError {
    fn into_response(self) -> Response {
        let (code, msg) = match self {
            ApiError::Invalid(m) => (StatusCode::UNPROCESSABLE_ENTITY, m),
            ApiError::NotFound => (StatusCode::NOT_FOUND, "order not found"),
        };
        (code, Json(serde_json::json!({ "error": msg }))).into_response()
    }
}

// ها به اشتراک می‌گذارند task مالکیت را بین Arc وضعیت مشترک
// RwLock دسترسی همزمان را ایمن می‌کند. کامپایلر بدون این دو اجازه اشتراک نمی‌دهد
type Store = Arc<RwLock<HashMap<String, Order>>>;

fn validate(req: &OrderRequest) -> Result<(), ApiError> {
    if req.customer_id.trim().is_empty() {
        return Err(ApiError::Invalid("customer_id is required"));
    }
    if req.items.is_empty() {
        return Err(ApiError::Invalid("at least one item is required"));
    }
    if req.items.iter().any(|i| i.qty <= 0) {
        return Err(ApiError::Invalid("qty must be positive"));
    }
    Ok(())
}

fn total(items: &[Item]) -> f64 {
    items.iter().map(|i| i.qty as f64 * i.unit_cost).sum()
}

async fn create_order(
    State(store): State<Store>,
    Json(req): Json<OrderRequest>,
) -> Result<(StatusCode, Json<Order>), ApiError> {
    validate(&req)?;

    let now = time::OffsetDateTime::now_utc();
    let order = Order {
        id: now.unix_timestamp_nanos().to_string(),
        customer_id: req.customer_id, // مالکیت منتقل شد، بدون کپی
        total: total(&req.items),
        created_at: now.to_string(),
    };

    // قفل نوشتن فقط تا پایان این بلاک نگه داشته می‌شود
    store.write().await.insert(order.id.clone(), order.clone());

    Ok((StatusCode::CREATED, Json(order)))
}

async fn get_order(
    State(store): State<Store>,
    Path(id): Path<String>,
) -> Result<Json<Order>, ApiError> {
    let guard = store.read().await;

```

```

    let order = guard.get(&id).cloned().ok_or(ApiError::NotFound)?;
    Ok(Json(order))
}

#[tokio::main]
async fn main() {
    let store: Store = Arc::new(RwLock::new(HashMap::new()));

    let app = Router::new()
        .route("/orders", post(create_order))
        .route("/orders/{id}", get(get_order))
        .with_state(store);

    let listener = tokio::net::TcpListener::bind("0.0.0.0:8080").await.unwrap();
    println!("listening on :8080");
    axum::serve(listener, app).await.unwrap();
}

```

نکات: ساختار `Store` بدون `Arc` و `RwLock` اصلاً کامپایل نمی‌شود — کامپایلر شما را مجبور می‌کند درباره اشتراک‌گذاری وضعیت بین `task`ها صریح باشید. خطاها با نوع `Result` در امضای تابع کدگذاری شده‌اند و عملکرد ؟ انتشار آن‌ها را اجباری می‌کند؛ نمی‌توانید سهواً خطایی را نادیده بگیرید. `req.customer_id` بدون کپی به `Order` منتقل می‌شود (`move` semantics). به محض خروج `order` از دامنه، حافظه‌اش قطعی آزاد می‌شود.

مقایسه مستقیم دو پیاده‌سازی

جنبه	Go	Rust
وابستگی خارجی	صفر، فقط کتابخانه استاندارد	چهار (<code>axum</code> ، <code>tokio</code> ، <code>serde</code> ، <code>time</code>)
خطوط کد	کمتر و مستقیم‌تر	بیشتر، به‌خاطر تعریف صریح نوع خطا
مدیریت خطا	با قرارداد، قابل نادیده‌گرفتن	اجباری در زمان کامپایل
اشتراک وضعیت	<code>sync.RWMutex</code> به‌صورت اختیاری	<code>Arc<RwLock<...>></code> اجباری از سوی کامپایلر
احتمال <code>data race</code>	ممکن، با ابزار <code>race</code> - قابل کشف در زمان اجرا	غیرممکن در کد <code>safe</code> ، در زمان کامپایل مسدود می‌شود
زمان کامپایل	چند ثانیه	چند ده ثانیه تا چند دقیقه در <code>build</code> کامل
حجم باینری	حدود ۱۰ تا ۱۵ مگابایت	حدود ۳ تا ۸ مگابایت با <code>build</code> نسخه <code>release</code>

۵. معیارهای انتخاب برای تیم‌های فنی

Go را انتخاب کنید اگر...

- سرویس‌ها عمدتاً I/O-محور هستند: API Gateway، CRUD، BFF، orchestration، سرویس‌های چسبی.
- می‌خواهید ده‌ها میکروسرویس را سریع بسازید و نگه دارید.
- تیم شما ترکیبی و در حال رشد است و باید افراد را سریع onboard کنید.
- ابزارهای اکوسیستم Cloud Native برایتان مهم است (Kubernetes، Docker، Prometheus، etcd همه با Go نوشته شده‌اند و کلاینت‌های درجه یک دارند).
- زمان رسیدن به بازار مهم‌تر از آخرین ۲۰٪ کارایی است.
- بودجه سخت‌افزاری دارید و می‌توانید با افزودن یک نمونه دیگر، مسئله را حل کنید.

Rust را انتخاب کنید اگر...

- دنباله تأخیر بخشی از SLA است: p99 یا p999 باید پایدار باشد و توقف GC غیرقابل قبول است.
- بار کاری CPU-محور است: رمزنگاری، کدک، پردازش تصویر یا ویدیو، موتور قیمت‌گذاری، پردازش استریم با حجم بالا، استنتاج مدل.
- heap بزرگ و پر از اشاره‌گر دارید (کش‌های چند میلیون رکوردی) – دقیقاً سناریویی که Discord را به مهاجرت واداشت.
- مصرف حافظه مستقیماً به هزینه ابری تبدیل می‌شود و در مقیاس چند هزار نمونه اجرا می‌شود.
- سرویس در لبه یا محیط محدود منابع اجرا می‌شود (edge، WebAssembly، IoT).
- الزامات امنیتی سخت‌گیرانه دارید و می‌خواهید کل رده آسیب‌پذیری‌های حافظه را در زمان کامپایل حذف کنید.

راهنمای ترکیبی (پیشنهاد عملی برای اکثر سازمان‌ها)

این الگو در عمل بهترین نسبت بازده به ریسک را دارد:

1. پیش‌فرض سازمان: Go. لایه سرویس، API، هماهنگ‌کننده‌ها، و کارهای پس‌زمینه معمولی.
2. Rust به صورت هدفمند فقط برای ۵ تا ۱۰ درصد سرویس‌هایی که پروفایل‌گیری نشان داده گلوگاه واقعی‌اند: مسیر داغ، کش بزرگ، پردازش سنگین.
3. معیار ورود به Rust را از قبل تعریف کنید (مثلاً: «اگر p99 بالای X میلی‌ثانیه ماند و تنظیم GOMEMLIMIT / GOGC و کاهش تخصیص جواب نداد»). هرگز بر اساس سلیقه مهاجرت نکنید.
4. قبل از بازنویسی، Go را تنظیم کنید. در بسیاری از موارد تنظیم GOMEMLIMIT، استفاده از sync.Pool، حذف تخصیص‌های اضافی در مسیر داغ، و به‌روزرسانی به نسخه‌ای با زباله‌روب Green Tea مسئله را بدون تغییر زبان حل می‌کند.

انتخاب کنید	اگر مهم‌ترین محدودیت شما این است...
Go	سرعت تحویل و اندازه تیم
Rust	p99 پایدار زیر بار سنگین
Rust	هزینه حافظه در مقیاس بزرگ
Go	سهولت استخدام و onboarding
Rust	کار سنگین CPU
Go	یکپارچگی با اکوسیستم Kubernetes
Rust	ایمنی حافظه تضمین‌شده در زمان کامپایل
Go	سرویس‌های ساده و پرتعداد CRUD
Rust	اجرا در محیط محدود منابع یا WebAssembly

۶. ریسک‌ها و ملاحظات پایانی

- **ریسک اصلی Go:** فرض اینکه GC هرگز مشکل نمی‌شود. تا زمانی که heap کوچک است درست است؛ در مقیاس کش بزرگ، این فرض شکسته می‌شود.
- **ریسک اصلی Rust:** دست‌کم‌گرفتن هزینه انسانی. اگر تیم آموزش، بازبینی کد و زمان کافی برای عبور از دو دیوار یادگیری نداشته باشد، سود عملکردی زیر بار تأخیر تحویل و خستگی تیم دفن می‌شود.
- **هر دو زبان** استقرار در کانتینر را ساده می‌کنند: باینری استاتیک، ایمج کوچک، بدون نیاز به زمان‌اجرای جداگانه.
- **پیش از هر تصمیم، اندازه‌گیری کنید.** یک نمونه اولیه از مسیر داغ خود را در هر دو زبان بنویسید و زیر بار واقعی با داده واقعی پروفایل بگیرید. بنچمارک‌های عمومی جهت را نشان می‌دهند، اما تصمیم شما باید بر اساس بار کاری خودتان گرفته شود.

منابع

• [Why Discord is switching from Go to Rust – Discord Engineering](#)

• [A Guide to the Go Garbage Collector – go.dev](#)

• [The Green Tea Garbage Collector – Go Blog](#)

- [Watching Go's new garbage collector move through the heap – The Consensus](#) •
- [Surviving the Rust Learning Curve – Rustify](#) •
- [Ownership – The Rust Programming Language](#) •
- [Stack Overflow Developer Survey 2025](#) •